Ders 5 Çalışma Özeti

Ders 5 Çalışma Özeti

Genel Konular

- Özyinelemeli (recursive) fonksiyonların mantığı ve ne zaman kullanılması gerektiği.
- Faktöriyelin iteratif ve özyinelemeli yazımı.
- Dizi elemanlarının toplamını özyinelemeli bulma.
- Özyineleme yazmanın adımları: temel durum (base case), özyinelemeli durum (recursive case), sonlanmanın garanti edilmesi.
- Doğrudan (direct) ve dolaylı (indirect) özyineleme.
- Özyinelemenin maliyeti: stack kullanımı, fonksiyon çağırma yükü (overhead), stack overflow.
- Kötü özyineleme örneği: Fibonacci'de aynı alt problemlerin tekrar tekrar hesaplanması.
- Özyinelemeli ikili arama (binary search).
- Hanoi Kuleleri (Towers of Hanoi) problemi (öğrencilere çözmeleri için bırakıldı).
- Bağlı listelere (linked list) giriş: statik dizi, dinamik bellek ayırma ve bağlı liste karşılaştırması.

Hocanın Özellikle Vurguladığı Kısımlar

- Bir problem döngüyle (iteratif) aynı açıklıkta çözülebiliyorsa döngü kullanılmalı. Özyineleme gerçekten gerektiğinde kullanılmalı.
- Quicksort, merge sort, ağaç ve graf algoritmaları gibi problemlerde özyinelemeli çözüm çok daha basit ve anlaşılırdır. Bu durumlarda özyineleme doğru seçimdir.
- Faktöriyel özyinelemeye örnek olarak gösterilir ama aslında iteratif yazmak daha uygundur.
- Sınavlarda her şeyi özyinelemeli yazmaya çalışan öğrenciler oluyor ve genelde hata yapıyorlar. "Sofistike görünsün" diye özyineleme kullanmayın.
- Her özyinelemeli fonksiyonda mutlaka bir temel durum olmalı. Yoksa fonksiyon sonsuza kadar kendini çağırır.
- Fibonacci örneği, alt problemleri birbirine bağlı olan bir problemde özyinelemenin kötü kullanımıdır.
- Özyineleme pratiği için önerilen alıştırmalar: Hanoi Kuleleri, bir sayının n'inci kuvveti, bir kelimenin palindrom olup olmadığı (kabak, yay gibi).
- Hanoi Kuleleri'nin çözümüne internetten bakmayın, kendiniz çözmeye çalışın. Demolarına bakılabilir.
- Pointer'lar `malloc` ile ayrılan yerde sıfırlanmaz. Değişkenleri ve pointer'ları her zaman kendiniz ilk değerle başlatın (initialize).

Kısa Tekrar Notları

- Özyineleme: Aynı fonksiyonu daha küçük bir problem için tekrar çağırmak.
- Temel durum: Özyinelemeyi durduran koşul.
- Özyinelemeli durum: Fonksiyonun kendini çağırdığı yer.
- Özyineleme yazma adımları:
  1. Temel durumu belirle.
  2. Özyinelemeli durumu belirle.
  3. Her çağrının sonunda temel duruma ulaşıldığından emin ol (ensure recursion terminates).

- Doğrudan özyineleme: Fonksiyon kendini çağırır. Dolaylı özyineleme: `f1` `f2`'yi, `f2` de `f1`'i çağırır.
- Her özyinelemeli çağrı stack'te fonksiyonun yeni bir kopyasını (parametreler, yerel değişkenler, dönüş adresi) oluşturur. Bu hem yer hem zaman maliyetidir.
- Özyinelemeli Fibonacci'de $F(6)$ için $F(1)$ 8 kez hesaplanır. Toplama sayısı yaklaşık 1.6^n hızıyla büyür ($F(6)$ için 12 toplama).
- İkili arama yalnız sıralı dizide çalışır. Her adımda aranan aralık yarıya iner.
- Bağlı liste avantajları: boyut sabit değil, ardışık bellek gerekmez, ekleme/silme kolay.
- Bağlı liste dezavantajları: rastgele erişim (random access) yok, her düğümde pointer için ek yer gerekir, önbellek (cache) verimi düşüktür.

Detaylı Açıklamalar

Faktöriyel: İteratif ve Özyinelemeli

Tanım:

$$n! = \begin{cases} 1 & n = 0 \\ n \cdot (n-1)! & n > 0 \end{cases}$$

- $4! = 4 \cdot 3! = 4 \cdot 3 \cdot 2! = 4 \cdot 3 \cdot 2 \cdot 1!$ şeklinde geriye doğru gidilir.
- $n!$'i hesaplamak için $(n-1)!$'e ihtiyaç vardır. Bu yapı özyinelemeye uygundur.

İteratif hâli (sayının pozitif verildiği varsayılıyor):

```
int factorial(int n)
{
    int i, result = 1;
    for (i = 1; i <= n; i++)
        result = result * i;
    return result;
}
```

- $4!$ için: $i = 1$: 1, $i = 2$: 2, $i = 3$: 6, $i = 4$: 24.

Özyinelemeli hâli:

```
int factorial(int n)
{
    if (n < 0)
        return -1;      /* negatif değer için hata */
    if (n == 0)
        return 1;       /* temel durum */
    else
        return n * factorial(n - 1);   /* özyinelemeli durum */
}
```

- Temel durum olmasaydı `factorial(3)` sırasıyla `factorial(2)`, `factorial(1)`, `factorial(0)`, `factorial(-1)`... diye sonsuza kadar giderdi. Döngüdeki sınırın görevini burada temel durum üstlenir.
- `factorial(3)` için çağrı ve dönüş sırası:

```
factorial(3) -> 3 * factorial(2)
  factorial(2) -> 2 * factorial(1)
    factorial(1) -> 1 * factorial(0)
      factorial(0) -> 1          (temel durum)
    factorial(1) = 1 * 1 = 1
  factorial(2) = 2 * 1 = 2
factorial(3) = 3 * 2 = 6  -> ana programa döner
```

Dizi Elemanlarının Toplamı

Dizi: `a = {3, 5, 7, 2}`, beklenen sonuç $3 + 5 + 7 + 2 = 17$.

```c
int sumOfArray(int a[], int i)
{
    if (i < 0)
        return 0;                                /* temel durum */
    else
        return a[i] + sumOfArray(a, i - 1);      /* özyinelemeli durum */
}

/* main içinde: result = sumOfArray(a, n - 1); */
```

İteratif karşılığı:

```c
int sumOfArray(int a[], int n)
{
    int i, sum = 0;
    for (i = 0; i <= n; i++)
        sum += a[i];
    return sum;
}
```

- İki fonksiyon aynı şekilde `n - 1` ile çağrıldığı için iteratif döngüde `i <= n` kullanıldı.
- Özyinelemeli izleme ($i = 3$ ile başlar, kısaca `s` yazıldı):

Çağrı	İfade	Dönen değer
`s(a, 3)`	`a[3] + s(a, 2)` = 2 + 15	17
`s(a, 2)`	`a[2] + s(a, 1)` = 7 + 8	15
`s(a, 1)`	`a[1] + s(a, 0)` = 5 + 3	8
`s(a, 0)`	`a[0] + s(a, -1)` = 3 + 0	3
`s(a, -1)`	temel durum	0

Özyineleme Türleri

- **Doğrudan özyineleme (direct recursion):** Fonksiyon kendini doğrudan çağırır. Örneğin `F(z)` içinde `F(z/2)` çağrılır.
- **Dolaylı özyineleme (indirect recursion):** Az kullanılır ve takibi zordur.

```c
int f2(int x);   /* f1, f2'yi tanımından önce çağırdığı için bildirim gerekir */

int f1(int x)
{
    if (x == 1)
        return x;
    else
        return f2(x);
}

int f2(int x)
{
    return f1(x);
}
```

- `f1` `f2`'yi, `f2` de tekrar `f1`'i çağırır. Böyle bir yapıda sonlanmanın çok dikkatli takip edilmesi gerekir.

Özyinelemenin Maliyeti

- Her özyinelemeli çağrıda fonksiyonun yeni bir kopyası belleğin stack bölgesinde oluşturulur. Parametreler, yerel değişkenler ve dönüş adresi burada tutulur.
- `factorial(3)` için ayrılan yer serbest bırakılmadan `factorial(2)` için yeni yer ayrılır, o bırakılmadan `factorial(1)` için yer ayrılır.
- **Stack overflow** hatasının olası sebepleri:

  - Özyinelemeli fonksiyon temel duruma hiç ulaşamıyordur.
  - Fonksiyon çok fazla sayıda kendini çağırmıştır.
  - Hoca, özyineleme olmadan da bir fonksiyonun sonsuz döngüde kalmasıyla bu hatanın görülebileceğini söyledi (bkz. ders sonrası notlar).
- Stack'i yönetmek (parametre aktarmak, dönüş adresi saklamak, yerel değişken oluşturmak) ek yük (overhead) getirir. Aynı işi yapan iki algoritmadan özyinelemeli olan genelde daha yavaştır.
- Özyineleme, çözümün bir önceki durumları bilmeyi gerektirdiği, yani stack gibi çalışan bir yapıya ihtiyaç duyulduğu problemlerde gerçekten gereklidir.

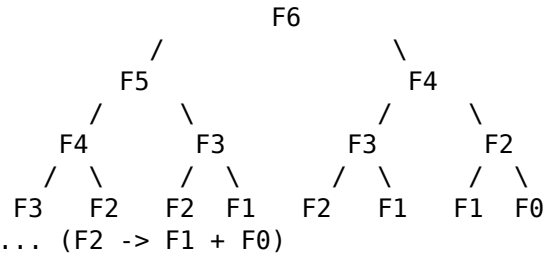
Kötü Özyineleme Örneği: Fibonacci

Fibonacci dizisi: 0, 1, 1, 2, 3, 5, 8, ... Her sayı kendinden önceki iki sayının toplamıdır.

$$F(n) = \begin{cases} 0 & n = 0 \\ 1 & n = 1 \\ F(n-1) + F(n-2) & n > 1 \end{cases}$$

```c
int fib(int n)
{
    if (n == 0)
        return 0;
    else if (n == 1)
        return 1;
    else
        return fib(n - 1) + fib(n - 2);
}
```

- Hocanın sorusu: Önceki `if`'ler zaten `return` ile çıkıyorsa son `else` gerekli mi?
  - Cevap: Gerekli değil, `else` olmadan da program doğru çalışır.
  - `else` yalnız okunabilirlik için yazılır. Kodu okuyan kişi "0 ve 1 değilse buraya girer" sonucunu hesap yapmadan görür.
- `fib(6)` çağrı ağacı (kısaltılmış):



- Aynı değerler defalarca hesaplanır: $F(4)$ 2 kez, $F(3)$ 3 kez, $F(1)$ 8 kez çağrılır.
- Hoca, $F(n)$ için yapılan toplama sayısının yaklaşık 1.6^n olduğunu söyledi. Bu bir büyüme hızıdır, tam sayı değildir: `fib(6)` için tam 12 toplama yapılır (bkz. ders sonrası notlar).
- Sonuç: Kod çok şık görünse de alt problemler birbirine bağlı olduğu için gereksiz yere tekrar hesaplama yapılır. Bu problem döngüyle, önceki iki değeri tutarak basitçe çözülebilir; özyinelemeli yazmak uygun değildir.

Özyinelemeli İkili Arama (Binary Search)

- Dizi sıralı olmalıdır (küçükten büyüğe veya büyükten küçüğe).
- Aranan aralığın tam ortasındaki elemana bakılır. Aranan sayı ortadakinden büyükse sol yarının hiçbir elemanına bakmaya gerek yoktur; aralık yarıya iner.

Örnek 1: `{3, 5, 7, 11, 13, 17, 21, 29}`, $n = 8$, $x = 29$ (dizide var):

left	right	mid	a[mid]	Karar
0	7	3	11	29 > 11, left = 4
4	7	5	17	29 > 17, left = 6

left	right	mid	a[mid]	Karar
6	7	6	21	29 > 21, left = 7
7	7	7	29	Bulundu

Örnek 2: Aynı dizi, $x = 8$ (dizide yok):

left	right	mid	a[mid]	Karar
0	7	3	11	8 < 11, right = 2
0	2	1	5	8 > 5, left = 2
2	2	2	7	8 > 7, left = 3
3	2			left > right, eleman yok

- Başlangıç indisi bitiş indisinden büyük olamaz. `left > right` olduğunda arama durdurulur.
- Hoca ikili aramanın da döngüyle yazılmasının daha verimli olduğu görüşünde. Yine de çok kullanıldığı ve anlaşılır olduğu için özyinelemeli hâli gösterildi.

Kod (özyinelemeli çağrıların başında `return` olmalıdır, bkz. ders sonrası notlar):

```
int binarySearch(int a[], int low, int high, int x)
{
    int mid;
    if (low > high)
        return -1;                                  /* temel durum 1: bulunamadı */
    mid = (low + high) / 2;
    if (x == a[mid])
        return mid;                                 /* temel durum 2: bulundu */
    else if (a[mid] > x)
        return binarySearch(a, low, mid - 1, x);    /* sol yarı */
    else
        return binarySearch(a, mid + 1, high, x);   /* sağ yarı */
}

/* İlk çağrı: low = 0, high = n - 1 */
```

- İki temel durum vardır: `low > high` (bulunamadı, -1) ve `a[mid] == x` (bulundu, `mid`).
- `a[mid] > x` ise `high = mid - 1` yapılır; `a[mid] < x` ise `low = mid + 1` yapılır.

Örnek 3: `{3, 5, 7, 9, 11}`, $x = 5$:

low	high	mid	a[mid]	Karar
0	4	2	7	7 > 5, sol yarı
0	1	0	3	3 < 5, sağ yarı
1	1	1	5	Bulundu, 1 döner

Hanoi Kuleleri (Ödev Niteliğinde Soru)

- A çubuğunda büyükten küçüğe dizilmiş 4 halka var: en altta kırmızı, üstünde pembe, yeşil ve en üstte mavi.
- Amaç: Tüm halkaları aynı sırayla B çubuğuna taşımak. C çubuğu geçici olarak kullanılabilir.
- Kurallar:
  1. Her seferinde yalnız bir disk taşınabilir.
  2. Küçük bir diskin üzerine asla büyük bir disk konamaz.
  3. Yalnız en üstteki disk hareket ettirilebilir; aradan disk çekilemez.
- Hoca soruyu derste çözmedi. Öğrencilerin kendi başlarına özyinelemeli olarak çözmesini istedi.

Bağlı Listelere Giriş

Neden Bağlı Liste?

- **Statik dizi:** Bir sınıfın not ortalaması için her öğrenciye ayrı değişken tanımlamak mümkün değildir. Dizi, aynı amaçla kullanılan birden fazla veriyi bir arada tutar. Ama boyut baştan sabittir; 5 öğrenci için de 500'lük yer ayrılır (static allocation).
- **Dinamik bellek ayırma (dynamic memory allocation):** Eleman sayısı çalışma anında öğrenilince o kadar yer ayrılır. Sistem ardışık bir blok verir (örneğin 1000 numaralı adresten itibaren). Program boyunca hep n elemanla çalışılacaksa en iyi çözüm budur.
- **Bağlı liste:** Eleman sayısı belli değilse, elemanlar geldikçe eklenip işi bitince silinecekse kullanılır. Ağaç ve graf yapılarında (örneğin her satırında farklı sayıda eleman olan komşuluk yapıları) çok kullanılır.

Düğüm Yapısı

```c
struct node {
    int value;
    struct node *next;
};

struct node *p;
p = (struct node *) malloc(sizeof(struct node));
if (p == NULL)
    /* yer ayrılamadı */;
```

- Listenin her elemanına düğüm (node) denir.
- Yapı kendine göndermelidir (self-referential): `next`, aynı yapıdaki başka bir düğümün adresini tutan pointer'dır. Hoca bunu "yapının kendisi özyinelemeli" diye ifade etti.
- `malloc` yer ayıramazsa `NULL` döndürür.
- **Head:** Listenin ilk düğümünü gösterir. Listeye her zaman head üzerinden erişilir.
- **Tek yönlü bağlı liste (singly linked list):** Her düğüm bir sonrakini gösterir. Yalnız ileriye doğru gidilebilir. Son düğümün `next`'i `NULL`'dır; `NULL`'a gelindiğinde listenin sonuna ulaşıldığı anlaşılır.

```
head -> [5 | 1250] -> [7 | 1300] -> [8 | NULL]
         (1000)        (1250)        (1300)
```

- Düğümler bellekte ardışık olmak zorunda değildir. Ayrıldıkları anda nerede boş yer varsa orada dururlar.

Temel İşlemler

- Ekleme (insert).
- Silme (delete).
- Arama (search).
- Tüm elemanları gösterme (display).

Avantajlar

- Liste uzunluğu sabit değildir.
- Ardışık blok gerekmez. Bellekte yeterince büyük ardışık boş yer yoksa bile kullanılabilir.
- Ekleme ve silme kolaydır:
  - Dizide ortadan bir eleman silinince ya yer boş kalır ya da sonraki tüm elemanlar kaydırılır.
  - Bağlı listede silinecek düğüm `free` ile bırakılır, önceki düğümün `next`'i silinenin sonrakini gösterir. Örneğin 1000 adresindeki düğüm 1250 yerine 1300'ü gösterir.
  - Araya ekleme de böyledir. 1400 adresindeki yeni düğüm için önceki düğüm 1400'ü, yeni düğüm de 1300'ü gösterir. Hiçbir eleman kaydırılmaz.

Dezavantajlar

- **Rastgele erişim yoktur.** Bir düğüme ulaşmak için baştan itibaren tek tek ilerlemek gerekir.
- **Ek yer gerekir.** Her düğümde bir pointer tutulur. Dinamik dizide ise tek bir pointer bloğun başını gösterir.
- **Önbellek verimi düşüktür.** (Hocanın açıklamasının düzeltmesi için ders sonrası notlara bakın.)

`malloc` ve `calloc`

- Bir öğrencinin sorusu: Pointer'ların varsayılan değeri `NULL` mu?
- Cevap: Pointer'ların varsayılan bir değeri yoktur.
  - `calloc` ayırdığı yeri sıfırlar.
  - `malloc` sıfırlamaz; o bellek bölgesinde önceden ne varsa (çöp değer, garbage) o kalır.
- Bu yüzden değişkenleri ve pointer'ları her zaman kendiniz başlatın. Her dilin bellek ayırma fonksiyonu aynı davranmayabilir; sıfırlanmıyormuş gibi davranmak en güvenlisidir.

Ders sonrası notlar

- **İkili aramada `return`:** Özyinelemeli çağrıların başında `return` olmalıdır. `return` yazılmazsa alt çağrının bulduğu indis çağırana iletilmez ve fonksiyon değer döndürmeden biter (C'de tanımsız davranış). `return` ile yazılan kod python karşılığıyla denendi: 29 için 7, 8 için -1, Örnek 3'te 5 için 1 döner.
- **`mid` hesabı:** `(low + high) / 2` çok büyük dizilerde `int` taşmasına yol açabilir. Güvenli yazım `low + (high - low) / 2`'dir.
- **Fibonacci sayıları:** Hoca $F(2)$'nin önce 4, sonra 5 ve 6 kez, $F(3)$'ün 2 kez çağrıldığını söyledi. `fib(6)` için doğru değerler (python ile sayıldı): $F(5)$ 1, $F(4)$ 2, $F(3)$ 3, $F(2)$ 5, $F(1)$ 8, $F(0)$ 5 kez. $F(1)$ için söylenen 8 doğrudur. Toplam toplama sayısı 12'dir; 1.6^n ifadesi yalnız büyüme hızını verir ($1.6^6 \approx 17$).
- **Stack overflow:** Özyineleme olmayan bir fonksiyondaki sonsuz döngü stack overflow'a yol açmaz; program sadece takılır. Stack overflow genelde sonlanmayan veya çok derin özyinelemeden ya da stack'te çok büyük yerel diziler tanımlanmasından kaynaklanır.
- **Önbellek:** Hoca bağlı listenin "heap'te tutulduğu ve öyle tasarlandığı için cache'te kullanılamadığını" söyledi. Bu doğru değil. Heap bölgesindeki veri de önbelleğe alınır. Asıl sorun düğümlerin bellekte dağınık olmasıdır: ardışık elemanlar aynı önbellek satırına düşmediği için sık önbellek ıskalaması (cache miss) olur. Dizide ise ardışık elemanlar birlikte önbelleğe alınır (locality).
- **`malloc` açıklaması:** Hoca "P kendisi 2500 adresinde duruyorsa orada ne varsa onu gösterir" dedi. Doğrusu: İlk değer verilmemiş bir pointer değişkeninin içinde çöp bir adres vardır. `malloc` ise geçerli bir adres döndürür ama o adresteki içeriği sıfırlamaz. Yeni düğümün `next` alanına `NULL` atanmalıdır.
- **İteratif faktöriyel:** Hoca döngü koşulunu sözlü olarak "i n'den küçük olduğu sürece" diye okudu. $i = 1$'den başlayan döngüde `i < n` son çarpanı atlar ve $4!$ yerine $3! = 6$ verir. Doğrusu yukarıdaki gibi `i <= n`'dir.
- **Hanoi Kuleleri:** n disk için en az $2^n - 1$ hamle gerekir; 4 disk için bu 15 hamledir.
- **Dizi toplamının iteratif hâli:** Fonksiyon `n - 1` ile çağrıldığı için döngü `i <= n` ile yazıldı. Fonksiyon eleman sayısıyla çağrılırsa döngü `i < n` olmalıdır.

Not: İsterseniz bu dersin altyazı (.srt) dosyasını Gemini Notebook gibi bir yapay zeka aracına yükleyerek ders hakkında daha detaylı soru-cevaplar yapabilir ve dersi verimli çalışabilirsiniz.